



MIAGE- L3

# Systèmes Informatiques

*Système d'exploitation et gestion de mémoire*

*Correction TD 3*

**BAKAYOKO Ibrahima**

**Enseignant-chercheur en informatique**

**UFR Mathématiques & Informatique**

[Bakayoko.ibrahima1@ufhb.edu.ci](mailto:Bakayoko.ibrahima1@ufhb.edu.ci)

Open Our Mind for a free world

# EXERCICE 1

## QUESTION

Un système d'exploitation exécute deux processus simultanément. Le processus A a décidé de stocker la variable FileName à l'adresse mémoire 1000. Au même moment, le processus B décide de stocker la variable ArrayIndex également à l'adresse 1000. Expliquez quelles opérations ont lieu lors d'un **changement de contexte** entre ces deux processus de façon à ce que les deux variables ne s'écrasent pas l'une l'autre.

Les adresses choisies par les différents programmes sont des adresses virtuelles.

Chaque programme peut choisir ses adresses virtuelles librement, il a accès à l'intégralité de l'espace d'adressage. Par contre, l'OS va établir un mapping entre chaque page de mémoire avec une page de mémoire physique.

Les deux pages virtuelles des deux programmes qui contiennent l'adresse virtuelle 1000 vont être mappées sur des pages physiques différentes.

Chaque fois que l'OS donne la main à un processus il reprogramme le MMU avec les correspondances virtuelles/physiques du processus concerné.

Lorsqu'un changement de contexte (ou changement de processus) a lieu entre les processus A et B, le système d'exploitation effectue plusieurs opérations pour garantir que les variables ne s'écrasent pas l'une sur l'autre. Ces opérations sont principalement liées à la gestion de la mémoire et à la préservation de l'intégrité des données.

Voici comment cela fonctionne :

**Sauvegarde des registres** : Lorsqu'un processus est interrompu pour permettre à un autre processus de s'exécuter, les valeurs de tous les registres de la CPU associées à ce processus sont sauvegardées. Cela garantit que les données du processus interrompu ne sont pas perdues et peuvent être restaurées plus tard.

**Chargement des registres du nouveau processus :** Ensuite, les valeurs des registres associées au nouveau processus sont chargées. Cela permet au processus B de reprendre son exécution à partir de l'endroit où il s'était arrêté.



**Gestion de la table de pages (si nécessaire) :** Si les processus A et B utilisent la mémoire virtuelle avec gestion de la table de pages, le système d'exploitation ajuste les tables de pages pour refléter le contexte du nouveau processus. Cela garantit que les adresses virtuelles utilisées par le processus B pointent vers les bonnes adresses physiques, y compris l'adresse 1000 pour ArrayIndex.

**Restauration du pointeur de pile (si nécessaire) :** Le pointeur de pile est ajusté pour refléter l'état du processus B. Cela garantit que les appels de fonction et les variables locales sont correctement gérés.

**Reprise de l'exécution :** Le processus B peut maintenant reprendre son exécution à partir de l'endroit où il s'était arrêté. Cela inclut la manipulation de la variable `ArrayIndex` à l'adresse mémoire 1000.

Les variables FileName et ArrayIndex ne se chevauchent pas, car chaque processus possède son propre espace mémoire virtuelle isolé. Le changement de contexte garantit que le système d'exploitation attribue les adresses physiques appropriées pour ces variables en fonction de l'espace mémoire virtuelle du processus en cours d'exécution.

En résumé, les opérations lors d'un changement de contexte garantissent que les variables de deux processus distincts ne s'écrasent pas l'une sur l'autre, car chaque processus a son propre espace mémoire isolé et les tables de pages sont ajustées en conséquence.

## EXERCICE 2

# QUESTION

Entre un processeur et son bus il se trouve deux éléments intermédiaires:

le MMU et le cache. Laquelle de ces deux organisations est correcte ? Expliquez pourquoi.

(a) CPU => Cache => MMU

(b) CPU => MMU => Cache

L'organisation (a) est incorrecte tandis que l'organisation (b) est correcte. Au-dessus du MMU les adresses mémoires manipulées sont des adresses virtuelles, alors qu'en-dessous on ne manipule que des adresses physiques. Si le cache mémoire stockait des adresses virtuelles, alors si deux processus utilisent la même adresse ils risqueraient après un changement de contexte de voir les variables de l'autre processus.

On pourrait demander au système d'exploitation de vider le cache mémoire lors d'un changement de contexte mais cela réduirait inutilement les performances en empêchant le cache de fonctionner normalement.



L'organisation correcte est la suivante :

(a) CPU => MMU => Cache

Cette organisation reflète généralement la séquence d'accès à la mémoire et aux données dans un système informatique moderne.

Voici pourquoi cette organisation est correcte :

**CPU (Unité Centrale de Traitement)** : L'unité centrale de traitement est responsable de l'exécution des instructions et du traitement des données. Elle communique directement avec le MMU (Unité de Gestion de Mémoire) pour accéder à la mémoire principale.

**MMU (Unité de Gestion de Mémoire) :** Le MMU est responsable de la gestion de la mémoire virtuelle et de la traduction des adresses virtuelles en adresses physiques. Il assure également la protection de la mémoire et le contrôle d'accès. Le MMU communique avec le CPU pour lui fournir les adresses physiques appropriées pour accéder à la mémoire.

**Cache** : Le cache est un composant intermédiaire entre la mémoire principale et le CPU. Son rôle est de stocker des données fréquemment utilisées pour accélérer l'accès à la mémoire. Lorsque le CPU accède à la mémoire, il vérifie d'abord si les données nécessaires se trouvent dans le cache. Si elles s'y trouvent (ce que l'on appelle un "hit de cache"), le CPU obtient rapidement les données depuis le cache, évitant ainsi un accès plus lent à la mémoire principale.

L'organisation "CPU => Cache => MMU" ne serait pas appropriée car le cache n'est généralement pas impliqué dans la gestion des adresses virtuelles et la traduction en adresses physiques. La gestion de la mémoire virtuelle et la traduction des adresses virtuelles en adresses physiques sont des tâches spécifiques du MMU.

Le cache est principalement lié à la performance en stockant des données pour accélérer l'accès à la mémoire, mais il n'est pas impliqué dans la gestion de la mémoire virtuelle.

Donc, l'organisation "CPU => MMU => Cache" reflète mieux le flux de données et de contrôle dans un système informatique typique.

## EXERCICE 3

Un ordinateur fonctionne de façon anormale, et les programmes semblent se planter de façon aléatoire alors qu'ils fonctionnent normalement sur d'autres machines. Après quelques recherches on découvre que la puce mémoire est endommagée et que l'octet mémoire à l'adresse 12 774 213 ne fonctionne plus : lorsqu'on lit cet octet on reçoit toujours 0 quelque soit la valeur qu'on y avait écrite.



Proposez une solution logicielle pour permettre à l'ordinateur de n'utiliser que les parties de la mémoire qui fonctionnent normalement.

Note : certains programmes sur cette machine (comme Skype par exemple) sont des logiciels propriétaires, et on ne peut pas les changer.

Il suffirait de faire en sorte que l'OS évite d'allouer la page de mémoire physique défectueuse (par exemple en la marquant comme déjà allouée).

Cela n'aura pas d'impact sur les processus qui ne voient que les adresses logiques.

Face à une puce mémoire défectueuse et à la nécessité de contourner ce problème sans modifier les logiciels propriétaires, vous pouvez envisager les approches ci-après :

**Isoler la mémoire défectueuse** : Si la position mémoire exacte du problème est connue, vous pouvez configurer le système d'exploitation pour qu'il n'utilise pas la plage mémoire endommagée. Cependant, cela peut être difficile à réaliser, car il faut connaître précisément les zones de mémoire à exclure, et cela peut varier selon les opérations et les programmes en cours d'exécution.

**Utilisation de la mémoire virtuelle :** Si votre système d'exploitation prend en charge la mémoire virtuelle, vous pouvez configurer une mémoire virtuelle plus grande pour compenser la perte de la mémoire physique. Cela permettra au système d'utiliser des emplacements de mémoire physique non endommagés.

**Utilisation de la mémoire ECC (Error-Correcting Code) :** Si votre matériel prend en charge la mémoire ECC, cela peut contribuer à détecter et corriger automatiquement les erreurs de mémoire. Cependant, cela dépend du matériel que vous avez et de la manière dont il est configuré.

**Développement de programmes de gestion des erreurs :** Vous pouvez développer des programmes (ou scripts) qui surveillent les opérations de lecture et d'écriture en mémoire et effectuent des vérifications pour s'assurer que les données en mémoire sont valides. Cela peut être un travail complexe et nécessite une compréhension approfondie du fonctionnement de la mémoire.

**Solutions tierces de gestion des erreurs :** Il existe des logiciels tiers conçus pour la gestion des erreurs matérielles, notamment les erreurs de mémoire. Ils peuvent vous aider à détecter et à gérer les erreurs de mémoire, même si cela ne résoudra pas le problème fondamental de la mémoire défectueuse.



**Remplacement de la puce mémoire défectueuse** : Si le matériel le permet, vous pouvez envisager de remplacer la puce mémoire défectueuse. Cependant, cela nécessite des compétences techniques avancées et peut ne pas être une option viable pour tous les types d'ordinateurs.

Dans tous les cas, il est essentiel de comprendre que travailler avec une mémoire défectueuse est complexe et peut entraîner des problèmes inattendus. Il est recommandé de faire des sauvegardes fréquentes des données importantes et de consulter un professionnel de l'informatique pour obtenir des conseils spécifiques à votre situation.

## EXERCICE 4

## QUESTION

Un ordinateur a une mémoire physique de 1 GB, et il exécute un processus qui utilise 1.5 GB. Expliquez ce qu'il se passe quand le programme essaye d'accéder à une variable qui n'est actuellement pas en mémoire vive :

- Que fait le système d'exploitation?
- Que fait le MMU?

La MMU déclenche une interruption de défaut de page vers le système d'exploitation. Le système d'exploitation choisit une page en mémoire à déplacer vers le disque (de préférence une page déjà stockée dans la zone d'échange afin d'éviter de la transférer à nouveau). Ensuite, il lit la page demandée depuis la zone d'échange vers la mémoire, et met à jour la table de correspondance des pages dans la MMU avant de reprendre le processus de niveau utilisateur.

La stratégie "Look" est une variante de la stratégie "SCAN" (ou "Elevator") utilisée pour ordonnancer les requêtes d'entrée/sortie sur un disque. Dans cette stratégie, le bras du disque se déplace dans une direction donnée jusqu'à ce qu'il atteigne la requête la plus éloignée dans cette direction, puis il revient en arrière sans traiter les requêtes en cours de route.

Lorsqu'un processus tente d'accéder à une variable qui n'est pas actuellement en mémoire vive (RAM), plusieurs opérations se produisent pour gérer la mémoire virtuelle. Cela implique à la fois le système d'exploitation (OS) et l'Unité de Gestion de Mémoire (MMU).

## **Action du système d'exploitation (OS) :**

Le système d'exploitation reçoit une notification de la MMU indiquant que le processus a tenté d'accéder à une page mémoire qui n'est pas actuellement en RAM. Cette situation est appelée un "page fault" ou "défaut de page".



Le système d'exploitation prend alors en charge le déroulement suivant. Il doit déterminer où se trouve la page mémoire requise (dans ce cas, la partie du processus qui n'est pas actuellement en RAM). Si la page mémoire n'est pas encore chargée en RAM, le système d'exploitation doit décider quelle page en RAM peut être libérée (évincée) pour faire de la place à la nouvelle page. La page évincée est ensuite écrite sur le disque si elle a été modifiée.

Ensuite, le système d'exploitation charge la page mémoire requise depuis le stockage (disque dur, SSD, etc.) vers la mémoire RAM. Cette opération est lente par rapport à l'accès à la mémoire RAM, ce qui peut entraîner des temps d'attente significatifs.

Enfin, le système d'exploitation met à jour les tables de correspondance des pages pour refléter le nouvel emplacement de la page mémoire en RAM. Le processus utilisateur peut alors reprendre son exécution, maintenant capable d'accéder à la variable.

**Action du MMU (Unité de Gestion de Mémoire) :** Le MMU joue un rôle clé dans la gestion de la mémoire virtuelle. Il est responsable de la traduction des adresses virtuelles du processus en adresses physiques de la mémoire RAM. Lorsqu'un processus tente d'accéder à une adresse virtuelle qui n'est pas en RAM, le MMU génère un signal de "page fault" (défaut de page) et signale cette condition au système d'exploitation.

Le MMU aide également à maintenir les tables de correspondance des pages (PT, Page Tables) qui indiquent quelles parties de la mémoire virtuelle du processus sont actuellement en RAM et où elles se trouvent.

En résumé, lorsqu'un processus tente d'accéder à une partie de la mémoire qui n'est pas actuellement en RAM, le système d'exploitation gère le déplacement de cette partie depuis le stockage vers la mémoire RAM, tout en libérant de la place en RAM en évinçant d'autres pages. Le MMU joue un rôle clé dans la traduction des adresses virtuelles en adresses physiques et signale les défaillances de page au système d'exploitation.

En résumé, lorsqu'un processus tente d'accéder à une partie de la mémoire qui n'est pas actuellement en RAM, le système d'exploitation gère le déplacement de cette partie depuis le stockage vers la mémoire RAM, tout en libérant de la place en RAM en évinçant d'autres pages. Le MMU joue un rôle clé dans la traduction des adresses virtuelles en adresses physiques et signale les défaillances de page au système d'exploitation.

## EXERCICE 5



Les pages mémoire d'un système d'exploitation ont traditionnellement une taille qui est une puissance de 2 : par exemple, une taille typique est 4096 octets. Serait-il possible d'utiliser une taille qui ne soit pas une puissance de deux, par exemple 5000 octets par page ? Quelles modifications faudrait-il faire sur notre machine ?

Il faut que le MMU soit capable de calculer efficacement le numéro de page et l'offset à l'intérieur de la page de n'importe quelle adresse virtuelle utilisée par le programme. Il faut donc avoir un circuit très efficace qui calcule le quotient et le reste de la division  $\text{adresse\_virtuelle} / \text{taille\_de\_page}$ .

Quand la taille d'une page est une puissance de 2 alors on peut les obtenir par extraction (les  $n$  premiers bits représentent le quotient, les  $m$  derniers représentent le reste). Sinon il faudrait un circuit de division en nombre entiers très efficace.

L'offset représente la position relative de l'adresse virtuelle à l'intérieur de la page. Il indique la position précise de l'octet au sein de la page, permettant ainsi de déterminer l'emplacement spécifique de la donnée recherchée.